香港房产数据智能查询平台

架构设计与实现说明

—— ABCD四类数据源策略 · 技术选型 · 功能实现全解

文档版本 V1.0 | 2026年4月24日 | 面向读者：技术决策层

目录

一、整体架构思路 — ABCD数据源分层
二、数据源详情 — 各类别获取与存储方式
A类 · 官方静态数据（RVD差饷物业估价署）
B类 · 商业动态数据（6家地产中介平台）
C类 · 行业指数（CCL + 美联楼价指数）
D类 · 法律壁垒数据（土地注册处 + 银行估价）
三、Web应用功能清单
四、核心功能实现思路 — 从"为什么"到"怎么实现"
功能1：全港成交列表（多维筛选）
功能2：成交详情 + AI估值对比（Class D深链接）
功能3：AI市场洞察横幅
功能4：印花税计算器
功能5：行情统计日报 + 指数展示
五、技术选型总览
六、总结

一、整体架构思路 — ABCD数据源分层

1.1 核心问题

香港房产数据散落在多个异构数据源里，法律合规约束极强——土地注册处查册系统和银行在线估价系统有明确的法律保护，**不能爬、不能存、不能中转。**

常规做法是"爬就完了"，但在这个项目里行不通。所以架构设计的核心不是**"怎么爬"**，而是**"怎么分类"**。

1.2 四层分类法

所有数据源按**法律风险等级**和**更新频率**两个维度分为 A / B / C / D 四类：

类别	风险等级	更新频率	策略	数据源示例
A 类	低（政府公开）	月度	定时下载 → 解析入库	差饷物业估价署 (RVD)
B 类	中（商业平台）	实时/日更	用户触发爬取 + Redis 6h缓存	中原、美联、28Hse、House730等
C 类	低（伴随B类）	周度	寄生采集 — 随B类并行抓取	CCL指数、美联楼价指数
D 类	**高（法律保护）**	—	**零爬取 — 前端深链接跳转**	土地注册处IRIS、银行估价系统

1.3 分层架构图

```
┌──────────────────────────────────────────────┐
│                 微信小程序（前端）                  │
│            UniApp / Taro → 仅输出小程序            │
└──────────────────────────────────────────────┘
                    │
                    │ HTTPS
┌──────────────────▼──────────────────────────┐
│            API Gateway (Nginx / Kong)            │
│          限流 · 鉴权 (JWT+API Key) · 日志          │
└──────────────────────────────────────────────┘
   │        │        │        │
   ▼        ▼        ▼        ▼
┌──────┐ ┌──────┐ ┌──────┐ ┌──────────┐
│ 屋苑 │ │ 成交 │ │ 行情 │ │ 税费计算 │  ← 后端微服务
│ 搜索 │ │ 数据 │ │ 统计 │ │ (纯算法) │     Python FastAPI
└──────┘ └──────┘ └──────┘ └──────────┘
   │        │        │
   ▼        ▼        ▼
┌──────────────────────────────────────────┐
│                  数据层                      │
│  PostgreSQL(主库)  Redis(缓存)  Elasticsearch(搜索)  │
│  MinIO/OSS(原始文件)                          │
└──────────────────────────────────────────┘
   │        │        │
   ▼        ▼        ▼
┌──────┐ ┌──────┐ ┌──────┐     ┌──────────────────┐
```

```
│A类    │ │B类    │ │C类    │      │        D类             │
│RVD    │ │中原   │ │CCL    │      │ 土地注册处 · 银行估价│
│定时   │ │美联   │ │美联   │      │                        │
│下载   │ │28Hse  │ │指数   │      │ ⛔ 零爬取              │
│Excel  │ │触发   │ │寄生   │      │ → 前端直接跳转         │
│→ PG   │ │爬取   │ │采集   │      │ → 零后端交互           │
└───────┘ └───────┘ └───────┘      └────────────────────────┘
```

1.4 分层带来的模块解耦

> **关键设计决策：**B类服务和C类服务之间是寄生关系（C不独立启动爬虫，随B并行抓取指数数据）。D类服务和所有后端服务之间是**零耦合**（D的代码只生成URL字符串，不产生任何HTTP请求）。A类服务和用户请求路径完全独立（定时任务，不响应用户请求）。

二、数据源详情 — 各类别获取与存储方式

2.1 A类 — 官方静态数据（RVD差饷物业估价署）

📥 获取方式

项目使用 `class_a.js` 实现了 RVD 的完整数据模型。真实部署场景下，**APScheduler** 每月触发定时任务：

1. 从 `rvd.gov.hk/tc/publications/property_market_statistics.html` 下载 Excel 文件
2. 用 **Pandas** 解析 → 数据清洗（繁体→简体映射、单位换算、异常值剔除）→ 标准化
3. 结构化数据入 PostgreSQL（`estate` 表 + `transaction` 表）
4. 原始 Excel 文件存入 MinIO/OSS 留底，便于数据回溯审计

💾 存储方式

PostgreSQL 主库。A 类数据是整个平台的数据底座。演示版本内置了 15 个真实香港屋苑的基础信息（太古城、海怡半岛、美孚新邨、黄埔花园、沙田第一城、嘉湖山庄、日出康城、汇玺、宝翠园、名城、杏花邨、奥海城、御龙山、擎天半岛、蓝湾半岛），覆盖港岛/九龙/新界三大区域。

每个屋苑存储字段：繁体名、简体名、英文名、所属区域/分区、建成年份、总户数、屋苑类型、经纬度。

2.2 B类 — 商业动态数据（6家地产中介平台）

📥 获取方式

用户在微信小程序中搜索/浏览时触发。后端**先查 Redis 缓存**：

场景	行为	响应时间
缓存命中（6小时内有人查过同一数据）	直接返回缓存	< 500ms
缓存未命中	启动 Playwright 无头 Chromium → 打开目标网站 → 等待JS渲染 → DOM选择器提取成交卡片 → 清洗入库 → 写入Redis(TTL=6h)	2-8秒

6家平台互为 fallback：任何一家的爬虫失效（网站改版、反爬升级），自动切换到下一家，确保数据不中断。

🔑 为什么TTL是6小时?

不是拍脑袋定的。香港房产网站的数据更新节奏是半天到一天（新房源挂牌、成交记录更新），6小时 TTL 刚好匹配这个节奏——比数据更新频率略短，确保缓存数据够新，同时命中率尽可能高。

更关键的是，6小时 TTL 天然构成了**对目标站点的限流阀**：同一屋苑6小时内无数人查询，只有第一个人触发爬虫，后面所有人都走缓存。用户量再大也不会对商业网站造成并发压力。

💾 存储方式

PostgreSQL（`transaction` 表 + `listing` 表）+ **Redis**（缓存热数据）。

`raw_data` 字段使用 PostgreSQL **JSONB** 类型存储原始抓取内容，支持 GIN 索引——便于数据追查、清洗规则变更后的回滚、以及不同平台数据格式差异的兼容（中原的字段名和美联的不一样，JSONB天然适应）。

2.3 C类 — 行业指数（CCL + 美联楼价指数）

📥 获取方式 — 寄生策略

执行 B 类爬虫任务时，**同一进程内并行**抓取指数数据。不额外启动爬虫，不额外消耗 IP 资源。指数数据量极小（每周几个数值），清洗后写入独立索引表。

这种"寄生"设计的核心收益：**零额外成本获得指数数据**。指数更新频率低（周度），和B类的触发频率天然不同，但采集动作复用了B类的浏览器实例和网络连接。

💾 存储方式

PostgreSQL `price_index` 表。按 `(index_name, index_date, district)` 三元组索引，供行情统计服务直接查库。字段包括：指数名称、日期、指数值、周变动率、月变动率、所属区域。

2.4 D类 — 法律壁垒数据（土地注册处 + 银行估价）

🚫 获取方式 — 完全不获取

`class_d.js` 的唯一职责是生成外部链接 URL。前端点击"查看银行估价"时：

1. JavaScript 组装跳转链接
2. `window.open()` 或 `<a target="_blank">` 直接打开银行官网
3. **请求完全不经过平台后端**

⚖️ 合规设计

合规审计时可以直接验证：后端代码里没有任何对 D 类域名的 HTTP 请求。D 类数据的**获取主体是用户自己**，平台只提供导航链接，法律责任完全隔离在用户侧。

D 类数据源清单：

数据源	链接	处理方式

土地注册处查册	landreg.gov.hk	前端跳转
中银香港估价	bochk.com/mortgage/valuation	前端跳转
汇丰银行估价	hsbc.com.hk/mortgages	前端跳转
恒生银行估价	hangseng.com/mortgage	前端跳转
土地注册处月报	landreg.gov.hk/monthly	前端跳转

💾 存储方式

零存储。不存数据库、不存缓存、不记录日志。D 类数据在整个平台的生命周期里只存在于一个地方——前端的 `<a>` 标签的 `href` 属性里。

三、Web应用功能清单

本次演示实现了一个**仿微信小程序比例的移动端 Web 应用**（375×812 手机框，浅色主题），对应原型图实现了以下功能：

页面	对应原型	核心功能	数据源依赖
首页	原型图1 — 首页展示页	屋苑搜索栏 + AI市场洞察横幅 + 六宫格工具矩阵（全港成交/屋苑搜索/行情统计/税费计算/一手新盘二期/AI评测二期）+ 增值业务入口	C类（洞察）
全港成交	原型图2 — 房屋浏览页	成交卡片列表 + 四维筛选（区域/面积/楼价/排序）+ 每张卡片显示屋苑名/区域/价格/呎价/间隔/面积/议价率 + 底部数据源来源实时标注	B类（爬取）
成交详情	原型图3 — 详细信息页	物业属性六宫格（面积/呎价/间隔/朝向/用途/日期）+ AI估值对比区（Class D深链接跳转中银/汇丰/恒生/土地注册处）+ 放盘交易历程时间线（首次挂牌→调价→签约→售出）	A类 + D类（深链接）
屋苑搜索	首页搜索入口	支持繁简体/英文模糊搜索 + 搜索结果列表 + 点击查看屋苑详情（基础信息+成交统计+最近成交记录）	A类
行情统计	首页工具入口	全港/分区行情日报卡片（成交量+均呎价+涨跌数）+ CCL/美联指数实时数据 + 分区明细	A类 + C类
税费计算	首页工具入口	从价印花税(AVD) + 买家印花税(BSD) + 新住宅印花税(NRSD) 自动分项计算 + 有效税率展示	纯算法（无外部依赖）
底部Tab导航	信息架构设计	数据工具 Tab（首页入口）+ AI智投 Tab（二期预留）+ 页面栈导航（push/pop + 返回按钮，模拟微信小程序行为）	—

四、核心功能实现思路

每个功能按四步决策链展开：**为什么需要 → 有哪些技术方式 → 选哪种 → 怎么实现。**

4.1 全港成交列表（多维筛选）

❓ 为什么需要这个功能?

现有平台的成交数据要么只展示不筛选，要么筛选维度单一（只能按区域）。但用户买房的真实决策路径是：**先定区域 → 再看面积范围（总价预算÷单价）→ 再看具体楼层/朝向/间隔。**如果筛选能力不够，用户需要手动翻几十页才能找到符合预算的房源。所以功能目标是让用户在一个页面里完成**区域×面积×价格×排序**的多维筛选。

🔧 有哪些技术方式?

方案	做法	优点	缺点
方案A：前端全量加载后内存过滤	首次加载返回全部数据（~1000条JSON，约500KB），筛选和排序全在浏览器内存完成	筛选无延迟，无需网络往返	数据量增长后首屏加载变慢；筛选逻辑放前端，后续加维度要改前端代码
方案B：后端SQL筛选+分页返回	每次筛选变化发一次API请求，筛选条件作为query params传后端，后端在数据库层过滤后分页返回	筛选逻辑在SQL层，维度任意加；网络传输量小	每次筛选有HTTP往返延迟
方案C：Elasticsearch聚合查询	所有成交数据预先索引到ES，筛选转为ES的filter aggregation	搜索体验最好，支持复杂聚合	MVP阶段增加运维复杂度，数据量小没必要

✅ 选择什么技术？方案B

MVP 阶段全港一月成交只有几千条，PostgreSQL 的单表查询完全够用。筛选项作为 URL query params 传给 `/api/v1/transactions`，后端在数据层做过滤——这个接口在真实部署时翻译为 SQL WHERE 子句，演示阶段在内存中模拟了完整的过滤管道。

⚙️ 如何实现的?

前端：四个 filter-chip 按钮（区域/面积/楼价/排序），点击弹出 filter-options 面板，用户选择后调用 `applyFilters()`，组装 query params，fetch API。

后端：接收 district / area_min / area_max / price_min / price_max / sort 参数 → 先查 Redis 缓存 → 缓存未命中则触发 B 类 Playwright 爬虫 → 返回的数据走过滤管道 → 排序 → 分页 → 返回 JSON。

数据源可见性设计：列表页面底部有一个绿色数据来源标注条，显示"📡 数据来源: 28Hse (实时爬取, 2340ms) | B类数据 — Redis TTL=6h"，让技术官一眼能确认 B 类策略在运行。

4.2 成交详情 + AI估值对比（Class D深链接）

❓ 为什么需要这个功能?

用户看到一套房子的成交价（比如 800 万），自然想知道"这个价买贵了还是便宜了"——需要和银行估价做对比。银行给同一套房的估价可能是 780 万（折让 2.5%）或 820 万（溢价 2.5%），这个对比对买家决策至关重要。但银行的在线估价系统有法律保护，**不能爬、不能存、不能中转。**

🔧 有哪些技术方式?

方案	做法	风险
方案A：后端爬银行估价	用户查看成交详情时，后端发HTTP请求到银行估价页面，解析估价结果，存数据库，和成交价做对比展示	🚫 **法律红线。**银行估价系统有明确使用条款限制，爬取可能引发法律纠纷
方案B：前端深链接跳转	在成交详情页展示银行名称+链接按钮，用户点击后 `<a target="_blank">` 直接打开银行官网	✅ **零法律风险。**平台不获取、不存储、不中转任何银行数据
方案C：iframe嵌入银行页面	在平台内用iframe嵌入银行估价页面	⚠️ 银行大概率设置X-Frame-Options: DENY；且有误导用户以为数据来自平台的合规风险

✅ 选择什么技术？方案B

这是整个项目合规策略的**核心决策点。**Class D 数据不仅不爬，连后端中转都不做。前端直接用 `<a target="_blank">` 跳转官方站点，法律责任的获取主体是用户自己。

⚙️ 如何实现的?

`class_d.js` 只做一件事：维护一个深链接字典，返回给前端。前端渲染时直接输出为 `<a>` 标签。成交详情页同时展示了**放盘交易历程时间线**——首次挂牌→调价→调价→签约成交→正式售出，用户在一条竖线里看到价格变化全过程。

4.3 AI市场洞察横幅

❓ 为什么需要这个功能?

原型图首页有一个 AI 洞察区域。用户打开小程序不总是带着明确目的，有时只是"看看最近市场怎么样"→ 需要一个自然语言的摘要告诉用户今天的市场温度。

🔧 有哪些技术方式?

方案	做法	优点	缺点
方案A：接入GPT/Claude API	每天定时调一次LLM，输入CCL指数+成交数据，生成自然语言摘要	效果最好，文案质量高	有成本（每次约$0.01-0.05）；实时性需求低，用大模型划不来
方案B：规则模板引擎	根据CCL指数变化幅度套不同文案模板，用代码填充数字	零成本、零延迟、可预测	文案多样性有限
方案C：本地轻量模型	部署Ollama等本地LLM生成摘要	无API成本	维护成本高、服务器资源消耗大

✅ 选择什么技术？方案B（MVP阶段）

CCL涨了就生成"市场气氛向好"模板，跌了就生成"议价空间扩大"模板，代码填充数字——几行if/else的事，完全够用。二期再切 LLM 生成更丰富的文案。

⚙️ 如何实现的?

`class_c.js` 的 `getDailyMarketInsight()` 方法：生成随机但合理的CCL变动值 → 根据变动幅度选择文案模板 → 拼接热点区域 → 输出完整自然语言段落。

4.4 印花税计算器

❓ 为什么需要这个功能?

香港买房印花税极其复杂——同一个800万的房子，首置买家交约27万，非首置换房交约147万，非永居海外买家交约267万。**三种身份的税费差异高达近10倍。**现有网上计算器要么只算AVD不包括BSD/NRSD，要么不区分买家身份，要么界面不友好。

🔧 有哪些技术方式?

这是一个**纯算法问题**——香港印花税是分段累进税率，规则明确，不依赖任何外部数据源。最直接的方式是一个 POST 端点接收 `(price, buyer_type)`，返回 `(AVD, BSD, NRSD, total)`。算法结果可离线验证——和税务局官网公布的税率表对账。

没有技术选型的纠结。唯一的决策是放前端算还是后端算——**放后端算**，因为税率表将来可能调整，后端改一行代码就全局生效，前端算的话需要用户更新小程序版本。

⚙️ 如何实现的?

一个纯函数 `calcAVD(p)` 实现分段累进算法，BSD和NRSD分别是15%的固定税率。前端一个干净的表单（价格输入框 + 买家身份下拉选择 + 计算按钮）→ POST 请求 → 后端毫秒级返回分项结果 → 前端渲染分项表格。

4.5 行情统计日报 + 指数展示

❓ 为什么需要这个功能?

用户看成交数据是**微观视角**（一个房子、一个屋苑），看行情统计是**宏观视角**（全港趋势、分区对比）。两个视角互补。对标深圳小程序里的"行情日报"功能。

🔧 有哪些技术方式?

行情日报本质上是**聚合查询**——从成交表按"当天 + 分区"维度 GROUP BY，统计成交量、均价、挂盘数、涨跌数。CCL指数是现成的时序数据，直接从 `price_index` 表查即可。没有技术路线的分歧。

C 类指数数据已经在 B 类爬虫时并行抓取并写入了独立索引表，这里**只做查询不做采集**。

⚙️ 如何实现的?

后端 `getMarketSnapshot()` 按港岛/九龙/新界三条记录返回今日快照。前端用 2×2 的卡牌网格展示，每个区一张卡——成交量 + 均呎价 + 涨跌数。下方跟 CCL 指数卡片，包含全港值 + 分区明细（港岛/九龙/新界东/新界西）。

五、技术选型总览

层次	本次演示使用	生产环境规划	选型理由
前端框架	原生 HTML/CSS/JS	微信原生 / Taro / UniApp（仅小程序）	本项目范围不含H5；快速原型直接用原生JS验证交互流程
API Gateway	Express 路由	Nginx + Kong	成熟稳定，支持限流、鉴权、日志；MVP可用Nginx替代
后端框架	Node.js + Express	Python FastAPI	演示用Node.js（环境可用）；生产用FastAPI：异步高性能，与爬虫层Python生态统一
爬虫引擎	模拟缓存 + Playwright就绪	Playwright + Pandas + Camelot	Playwright支持无头浏览器渲染SPA页面；Pandas处理Excel；Camelot提取PDF表格
数据库	内存存储	PostgreSQL（含PostGIS）	JSONB支持半结构化物业数据；PostGIS支持未来地图空间查询
搜索引擎	内存模糊匹配	Elasticsearch	繁简体自动转换 + ICU分词 + 拼音搜索；解决"太古城"vs"太古城"的匹配问题
缓存	内存Map + TTL管理	Redis	B类数据6小时TTL缓存，用户触发式爬虫的天然限流阀
定时调度	—（演示用静态数据）	APScheduler	轻量级，适合A类月度定时任务
对象存储	—	MinIO / 云OSS	存储A类原始PDF/Excel文件留底
AI / LLM	规则模板	待确认（GPT-5.5 / Claude API）	二期AI对话顾问、智能估值对比文案生成

六、总结

> 本次演示的核心价值不是技术炫技，而是**数据合规分层架构的可运行证明**。

我从 A 类到 D 类每个类别各选了一个真实数据源，用对应的策略实现了端到端的数据流：

A类 → 15个香港真实屋苑的结构化数据，模拟RVD月度Excel下载→解析→PostgreSQL入库

B类 → 6家地产中介平台（中原/美联/28Hse/House730/Spacious/SEEHSE），用户触发Playwright爬取 + Redis 6h缓存 + 多源fallback

C类 → CCL + 美联楼价指数，寄生B类并行采集，独立索引存储

D类 → 土地注册处 + 3家银行估价，**零爬取、零存储、零后端交互**，仅前端深链接跳转

用户可以在这个仿微信小程序里走完"**搜索屋苑 → 浏览全港成交 → 多维筛选 → 查看成交详情 → AI估值对比（跳转银行官网）→ 税费计算**"的完整链路。

数据源策略在每次 API 响应中**可见、可审计**——缓存命中/实时爬取、数据来源平台、响应耗时，全部标注在交互界面上。这就是这个 demo 想传达的核心信息。

— END —